

Java Type Casting

Converting values between different data types

In Java, **Type Casting** is when you assign a value of one primitive data type to another type. It is essential for arithmetic precision and logic compatibility.

Widening Casting

Automatic: Smaller → Larger

Example: int to double

No data loss. Java does it for you.

Narrowing Casting

Manual: Larger → Smaller

Example: double to int

Requires parentheses. May lose data.

1. Widening (Automatic)

```
int myInt = 9;
double myDouble = myInt; // Automatic

System.out.println(myDouble); // Outputs 9.0
```

2. Narrowing (Manual)

⚠ Warning: Narrowing casting effectively "chops off" extra information. Converting 9.78 (double) to an int results in 9. The decimal part is discarded.

```
double myDouble = 9.78;
int myInt = (int) myDouble; // Manual cast

System.out.println(myInt); // Outputs 9
```

3. Real-Life Logic: Game Score %

Casting is crucial in division. Without casting, $423 / 500$ would result in 0 (integer division). Cast to double to keep the decimal!

```
int maxScore = 500;
int userScore = 423;

// Force double precision
double percentage = (double) userScore / maxScore * 100.0;

System.out.println("Score: " + percentage + "%");
```

Result: 84.6%

Recap

- **Type Casting:** Changes the data type of a value.
- **Automatic (Widening):** byte → short → char → int → long → float → double.
- **Manual (Narrowing):** double → float → long → int → char → short → byte.
- **Parentheses:** Always required for manual narrowing, e.g., (int) myVar.